

Tài liệu tham khảo có chú giải

TÓM TẮT

Danh sách *Nguồn* ở cuối sách liệt kê đầy đủ mọi tài liệu được trích dẫn, xếp theo thứ tự xuất hiện. Phụ lục này làm việc khác: nó nhóm các tài liệu theo *mục đích sử dụng*, nói rõ mỗi quyển dùng để làm gì, đọc vào lúc nào, và đọc bao nhiêu là đủ. Nếu chỉ nhớ một điều: chọn *một* quyển làm trục chính theo mục tiêu của bạn và dùng những quyển còn lại để tra — đọc song song ba giáo trình là cách chắc chắn nhất để không đọc xong quyển nào.

A. Bốn quyển làm trục chính — chọn một theo mục tiêu

Tự học tổng quát, muốn nhận diện được bài toán: Skiena, *The Algorithm Design Manual*[1]. Nửa đầu dạy kỹ thuật, nửa sau là danh mục tra ngược “bài của tôi trông thế này thì nó tên là gì”. Đây là quyển hợp nhất với người đi làm. Đọc nửa đầu tuần tự, nửa sau chỉ tra.

Muốn học cách nghĩ ra thuật toán: Kleinberg và Tardos, *Algorithm Design*[2]. Điểm mạnh riêng: mỗi kỹ thuật được dẫn ra từ một bài toán thật, và phần tham lam cùng phần luồng là phần trình bày tốt nhất mà tôi biết. Đọc chương 4, 6, 7 trước.

Thi đấu: Laaksonen, *Guide to Competitive Programming*[3] làm trục chính; Halim và cộng sự, *Competitive Programming 4*[4] làm danh mục kỹ thuật kèm bài luyện. Đọc kèm bộ bài CSES[5] theo đúng thứ tự chủ đề.

Muốn nền chứng minh vững, đặc biệt về đệ quy: Erickson, *Algorithms*[6], bản điện tử miễn phí. Phần quay lui và quy hoạch động dạy cách viết chứng minh song song với thuật toán — đúng tinh thần chương A/04.

B. Tra cứu, không đọc tuần tự

Cormen và cộng sự, *Introduction to Algorithms*[7]: tham chiếu chuẩn của ngành. Dùng để tra định nghĩa chính xác và chứng minh đầy đủ. Đọc tuần tự gần như luôn thất bại; đây không phải lỗi của bạn mà là do sách được thiết kế cho vai trò khác.

Knuth, *The Art of Computer Programming*[8], [9]: phân tích tới hằng số, độ sâu không sách nào bằng. Dùng khi cần hiểu tường tận *một* chủ đề cụ thể. Bài tập có xếp hạng độ khó 0–50; hãy chọn theo mức thay vì làm hết.

Sedgewick và Wayne, *Algorithms*[10]: mạnh về cài đặt chạy được và về thực nghiệm. Hợp khi bạn muốn thấy code thật kèm số đo thật.

Graham, Knuth, Patashnik, *Concrete Mathematics*[11]: tra khi phân tích độ phức tạp đựng tổng hoặc truy hồi khó (A/05).

cp-algorithms[12] và *KACTL*[13]: cài đặt ngắn gọn đã được kiểm chứng. Dùng để *đối chiếu* sau khi tự viết, không phải để chép.

C. Tư duy giải bài và cách luyện

Pólya, How to Solve It[14]: mỏng, đọc trong một buổi, và nên đọc trước mọi thứ khác. Nền của chương A/01.

Zeitz, The Art and Craft of Problem Solving[15] và *Engel*, Problem-Solving Strategies[16]: luyện tư duy bằng bài toán olympiad. Đọc khi muốn nâng khả năng nghĩ chứ không phải học thêm thuật toán.

Schoenfeld, Mathematical Problem Solving[17]: nghiên cứu thực nghiệm về quá trình giải bài; nguồn của khái niệm tự giám sát ở A/01.

Ericsson và Pool, Peak[18] cùng bài báo gốc năm 1993[19]: cơ sở của toàn bộ chương E/28. Đọc bài báo nếu muốn phần lỗi; đọc sách nếu muốn ví dụ.

Lehman, Leighton, Meyer, Mathematics for Computer Science[20]: nền toán rời rạc viết cho người làm tin học, bản điện tử miễn phí. Chương “State Machines” là nguồn của khung bất biến bảo toàn ở A/04; các chương về quy nạp và tổ hợp thay thế được phần lớn nội dung của A/05.

Bentley, Programming Pearls[21]: đọc rải rác, mỗi lần một chương ngắn. Là quyển gần nhất với công việc thật trong danh sách này.

Norvig, “Teach Yourself Programming in Ten Years”[22]: một bài viết ngắn, đọc mười phút, chỉnh lại kỳ vọng về tốc độ tiến bộ.

D. Chuyên sâu theo miền

Độ khó và NP: Garey và Johnson[23] vẫn là danh mục bài toán NP-đầy đủ đầy đủ nhất. Dùng khi cần tra “bài này đã biết là khó chưa”.

Ngẫu nhiên hoá: Motwani và Raghavan[24] cho chiều sâu; Mitzenmacher và Upfal[25] dễ vào hơn và dạy luôn phần xác suất cần thiết.

Xấp xỉ: Williamson và Shmoys[26] cho lý thuyết hiện đại; Vazirani[27] ngắn gọn hơn.

Tham số hoá: Cygan và cộng sự[28] — tài liệu chuẩn cho hướng “khó theo n nhưng dễ theo k ” ở D/23.

Hình học tính toán: de Berg và cộng sự[29]; đọc kèm cảnh báo về số thực ở D/22.

Cấu trúc dữ liệu nâng cao: khoá học công khai của Demaine[30] cho cây bền vững, cấu trúc thời gian, và các kết quả gần biên.

Luồng và mạng: Ahuja, Magnanti, Orlin[31] cho chiều sâu vận trù học.

Cấu trúc bất biến: Okasaki[32] nếu bạn làm việc với ngôn ngữ hàm hoặc cần cấu trúc bền vững.

E. Bài báo gốc đáng đọc trực tiếp

Nhiều kết quả trong sách này dễ hiểu hơn khi đọc bản gốc, vì tác giả nói rõ *vì sao* họ cần nó.

Hoare về Quicksort[33] và về cơ sở tiên đề cho chương trình[34]; Dijkstra về đường đi ngắn nhất[35] — hai trang, đọc trong mười phút. Tarjan về duyệt sâu[36] và về hợp nhất tập hợp[37]; Knuth về “tối ưu hoá sớm”[38] — đọc để thấy câu đó bị trích cụt thế nào. Cook[39] và Karp[40] cho khởi đầu của lý thuyết NP. Bentley và McIlroy về kỹ thuật hoá một hàm sắp xếp[41] — tài liệu tốt nhất về khoảng cách giữa thuật toán trong sách và hàm dùng được. Frigo và cộng sự về

cache-oblivious[42]. McSherry và cộng sự về chi phí của khả năng mở rộng[43] — ngắn, và thay đổi cách bạn đọc mọi báo cáo hiệu năng.

Từ đợt bổ sung 2026-09-07, mỗi chương có mục chuyên sâu đều kèm khối **Đọc thêm** riêng với đầy đủ nơi công bố và năm; danh sách dưới đây chỉ gom lại những công trình mới thêm, để tiện tra theo chủ đề.

Cấu trúc dữ liệu. Fredman và Tarjan về Fibonacci heap (*Journal of the ACM*, 1987) đọc kèm ngay với Larkin, Sen, Tarjan (*ALLENEX*, 2014) — một bên là cận, một bên là số đo, và đọc cùng lúc thì thấy rõ khoảng cách giữa hai thứ[44], [45]. Sleator và Tarjan về splay tree (*Journal of the ACM*, 1985) đọc kèm Demaine và cộng sự về Tango tree (*SIAM Journal on Computing*, 2007) cho câu hỏi tối ưu động[46], [47]. Pagh và Rodler về băm cuckoo (*Journal of Algorithms*, 2004) và Pătraşcu cùng Thorup về băm bảng tra (*Journal of the ACM*, 2012) cho phía bảo đảm của bảng băm[48], [49]. Bentley và Saxe (*Journal of Algorithms*, 1980) cho cách động hoá một cấu trúc tĩnh, và Fenwick (*Software: Practice and Experience*, 1994) cho bài gốc của cây chỉ số nhị phân[50], [51].

Giới hạn và cận dưới. Tarjan (*Journal of Computer and System Sciences*, 1979) cùng Fredman và Saks (*STOC*, 1989) cho biết “ α là tối ưu” nghĩa là gì trong từng mô hình[52], [53]. Backurs và Indyk (*SIAM Journal on Computing*, 2018; bản hội nghị *STOC* 2015) và Gajentaan cùng Overmars (*Computational Geometry*, 1995) là hai cửa vào tốt nhất cho độ phức tạp tĩnh[54], [55].

Biên giới hiện nay. Harvey và van der Hoeven về phép nhân $O(n \log n)$ (*Annals of Mathematics*, 2021); Agrawal, Kayal, Saxena về “PRIMES is in P” (*Annals of Mathematics*, 2004); Duan và cộng sự về việc phá rào cản sắp xếp cho đường đi ngắn nhất (*STOC*, 2025); Chen và cộng sự về luồng cực đại trong thời gian gần tuyến tính (*FOCS*, 2022)[56]–[59]. Bốn bài này chỉ nên đọc phần mở đầu — ở đó các tác giả kể lại lịch sử cận trên của bài toán, và đó mới là phần có giá trị lâu dài cho người học.

Quy hoạch động và hình học. Yao (*SIAM Journal on Algebraic and Discrete Methods*, 1982) cho bất đẳng thức tứ giác, Hirschberg (*Communications of the ACM*, 1975) cho truy vết trong bộ nhớ tuyến tính, Aggarwal và cộng sự (*Algorithmica*, 1987) cho SMAWK, Shewchuk (*Discrete & Computational Geometry*, 1997) cho vị từ hình học tin cậy[60]–[63].

F. Nguồn bài tập

CSES Problem Set[5]: có thứ tự, phủ đúng các chủ đề của phần B và C, và là nơi tôi lấy phần lớn bài trong ngân hàng ở E/28. Nếu chỉ chọn một nguồn, chọn nguồn này.

Chuỗi bài quy hoạch động của AtCoder: 26 bài phủ gần hết các họ trạng thái ở C/16.

Codeforces: bài phân loại theo mức và có thi ảo — nguồn tốt nhất để luyện dưới sức ép (E/27).

LeetCode: hợp cho chuẩn bị phỏng vấn hơn là cho thi đấu; dùng kèm McDowell[64] trong 4–6 tuần trước buổi phỏng vấn, không sớm hơn.

Skiena và Revilla, Programming Challenges[65]: bài cũ nhưng phân loại tốt và có phần bình luận cách nghĩ.

G. Chọn nguồn thế nào cho khỏi hỏng cây kiến thức

Một nguồn sai đưa vào cây tri thức sẽ âm thầm làm hỏng mọi thứ dựng lên trên nó, và vài năm sau bạn sẽ không nhớ nổi ý sai đến từ đâu. Vì vậy quyển này giữ một quy tắc chọn nguồn cố định.

Nguồn được tin: bài đã qua phản biện ở hội nghị hoặc tạp chí đầu ngành — với thuật toán là *Journal of the ACM*, *SIAM Journal on Computing*, *Algorithmica*, *Journal of Algorithms*, và các kỷ yếu STOC, FOCS, SODA, ALENEX; với phần toán là các tạp chí toán chuẩn — cùng các sách giáo trình đã qua nhiều lần tái bản. Ưu tiên thêm cho bài có mã nguồn công khai hoặc đã được người ngoài chạy lại.

Nguồn bị loại: hội nghị và tạp chí săn phí, bản tiền ấn chưa ai kiểm chứng và không kèm mã nguồn, bài chỉ khoe số đo mà giấu điều kiện thí nghiệm, và nội dung do mô hình ngôn ngữ sinh ra rồi đăng lại. Khi buộc phải dẫn một nguồn chưa qua phản biện — trong quyển này chỉ có đúng một chỗ, là con số mới nhất của số mũ nhân ma trận ở C/14 — thì nó được ghi rõ là tiền ấn ngay tại chỗ.

Một lưu ý riêng cho tài liệu lập trình thi đấu. cp-algorithms, KACTL và các blog kỹ thuật[12], [13] rất hữu ích và tôi dùng chúng thường xuyên, nhưng chúng là tài liệu cộng đồng, không qua phản biện. Cách dùng đúng: lấy chúng làm bản đối chiếu *cài đặt*, và lấy bài báo gốc làm nguồn cho *phát biểu và điều kiện áp dụng*. Rất nhiều lỗi “thuật toán này chạy trong $O(\cdot)$ ” lan truyền trong cộng đồng chỉ vì một điều kiện trong bài gốc bị bỏ quên khi viết lại.

CẦN NHỚ

Thứ tự đọc gợi ý nếu bạn bắt đầu từ đầu: Pólya (một buổi) → quyển trực chính theo mục tiêu → giải bài song song ngay từ tuần đầu → tra CLRS hoặc cp-algorithms khi vướng → đọc các bài báo gốc ở mục E khi đã quen với chủ đề tương ứng. Đừng đọc quá hai quyển cùng lúc.